HW4 Experimental Report
CNN-based Three-class Face Image Classification

羅政昕 **R37141329**

1. Report Coverage

The HW4 requires a CNN implementation for three-class classification, dataset preparation with the fixed class names hao, jin, and gua, at least three hyperparameter experiments, loss/accuracy analysis, selection of the best-performing model, trained weights, and prediction examples. This report covers those items.

Required Item	Where Addressed
Dataset collection and preprocessing	Covered in Sections 2-3
At least three experiments	baseline64, compact128, deeper160
Clear result table	Section 5
Loss / accuracy analysis	Section 6
Best model justification	Section 7
Testing / prediction examples	Section 8

2. Project Structure & How To Run

Project structure:

project_folder/

```
├── train.py
├── test.py
├── hidden_test
├── models
├── outputs
├── data/
│   ├── hao/
│   ├── jin/
│   └── gua/
└── data_face_only/
    ├── hao/
    ├── jin/
    └── gua/
```

Folder/File	Description
train.py	The training py file.
test.py	The testing (hidden test) py file.
hidden_test	Test images for testing/hidden test.
models	All the models are saved here, “best_model.pth” is the best one.
outputs	All the results outputs from the training and testing will be here.

	Training outputs: modelName_accuracy.png, modelName_loss.png, history_modelName.csv, prediction_examples_modelName.csv, prediction_examples_modelName.png **Testing outputs:** test_prediction.csv, test_prediction_examples.png
data	
data_face_only	

How to run:

Actions	Commands

train	**Run all experiments (3 models):**
	`python -u train.py --raw_dir . --run_all`
	
	Run the best (deepest) model and create the final best model:
	`python -u train.py --raw_dir . --config deeper160 --final_train_all --epochs 100`

test (hidden test)	**Run test with the best model:**
	`python test.py --data_dir hidden_test --weights models/best_model.pth --tta`
	(--data_dir [hidden test data route], change accordingly if needed.)
	
	If the TTA step is not allowed for the hidden test, run:
	`python test.py --data_dir hidden_test --weights models/best_model.pth`

3. Dataset and Split Design

The raw dataset contains both full-size/full-body images in data/ and face-only images in data_face_only/. Both folders were used. This is important because the hidden test may contain either face-only crops or full-body images, so to make a more generalized model and achieve the best result in the hidden test, I make sure that the model is trained on all kinds of images. The class folders are fixed as hao, jin, and gua.

The split manifest contains 240 image records. For data/ folder, I had collected 40 images for each person, and for data_face_only/ folder, I had cropped the face parts of the same images from the data/ folder. So 3 people * 40 images each * 2 types (full/face) = 240 images.

The split manifest shows a balanced split: 168 training images, 36 validation images, and 36 test images. Each class has 56 training images, 12 validation images, and 12 test images.

Split	gua	hao	jin	Total
train	56	56	56	168
val	12	12	12	36
test	12	12	12	36

The source-level split below confirms that both data/ and data_face_only/ were used in every class and split.

Split	Source	gua	hao	jin	Total
train	data	28	28	28	84
train	data_face_only	28	28	28	84
val	data	6	6	6	18
val	data_face_only	6	6	6	18
test	data	6	6	6	18
test	data_face_only	6	6	6	18

4. Preprocessing and Augmentation

- Image loading: images are opened with EXIF-aware RGB conversion to handle phone images and mixed file formats.
- Resize strategy: images are letterbox-resized to the configured input size. This keeps the whole original image and avoids destructive center cropping or any other type of cropping.
- Cropping policy: for configs with add_face_crop_view=True, a detected face crop may be added as an extra training view, but it does not replace the full image.
- Normalization: pixels are converted to tensors and normalized to the range [-1, 1].
- Training-only augmentation: flip, brightness/contrast changes, mild noise, and random erasing are applied only to the training dataset after splitting.
- Leakage control: full-body and face-only images with the same stem are kept in the same split by group-aware splitting.

5. Model Configurations

Config	Input Size	Model	Architecture Summary	Optimizer	LR	Dropout	Extra Face Crop View
baseline64	64 x 64	BaselineCNN	3 Conv-ReLU-MaxPool blocks + FC head	Adam	1e-3	0.50	No
compact128	128 x 128	CompactFaceCNN	ConvBN blocks + AdaptiveAvgPool + Dropout	AdamW	8e-4	0.35	Yes
deeper160	160 x 160	DeeperFaceCNN	Deeper ConvBN blocks + AdaptiveAvgPool + Dropout	AdamW	6e-4	0.40	Yes

Note: Extra Face Crop View is a mode that will add extra face crop images to the data if a face is detected by the OpenCV package. With this feature, I aim to increase the amount of face crop data.

6. Experimental Results

The following table was calculated from the corrected local history CSV files. The detailed histories are treated as the source of truth because they contain the per-epoch metrics actually produced by training.

Config	Epochs	Best Val Epoch	Best Val Acc	Val Loss	Train Acc	Test Acc @ Best Val	Best Test Epoch	Best Test Acc	Final Val Acc	Final Test Acc
baseline64	31	13	72.22%	0.8862	76.79%	61.11%	23	69.44%	63.89%	61.11%
compact128	60	39	83.33%	0.6978	88.69%	63.89%	53	80.56%	80.56%	75.00%
deeper160	62	44	86.11%	0.7039	94.05%	77.78%	50	80.56%	80.56%	75.00%

Best validation result: deeper160 at epoch 44 with 86.11% validation accuracy.

Best held-out test accuracy observed: compact128 and deeper160 both reached 80.56% on the experiment test split.

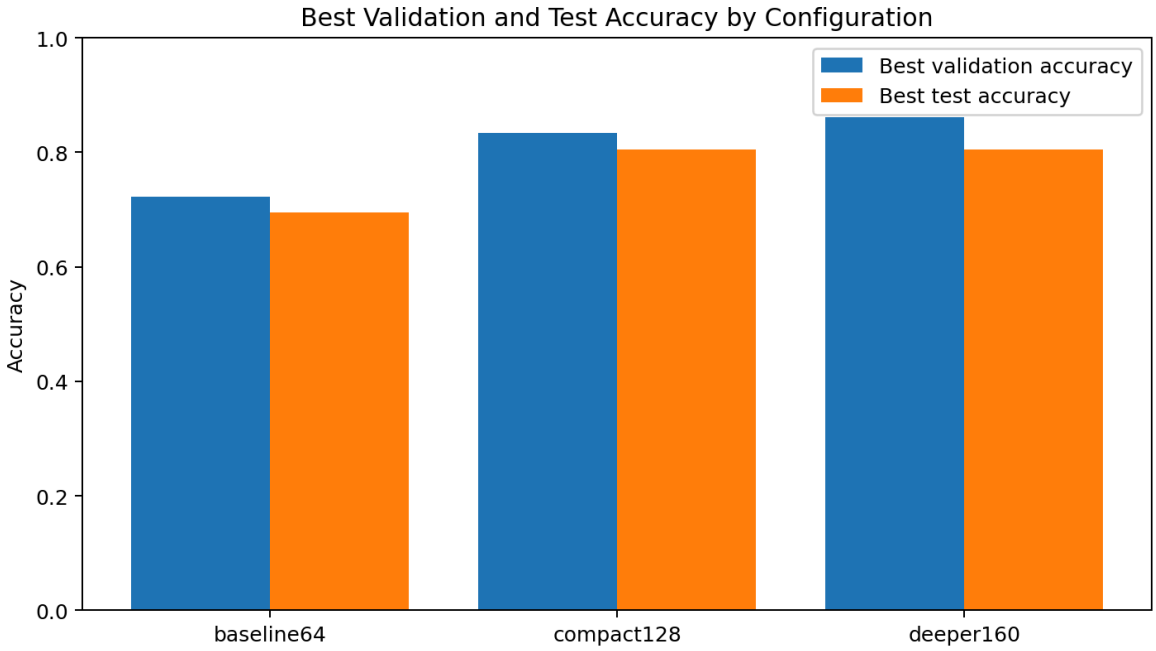

Figure 1. Best validation and test accuracy comparison.

7. Loss and Accuracy Curves Analysis

The figures below are the corrected local training plots. They show that training accuracy generally increases as loss decreases, but validation and test curves fluctuate because the validation/test sets contain only 36 images each. A few misclassified images can move the validation/test accuracy by several percentage points.

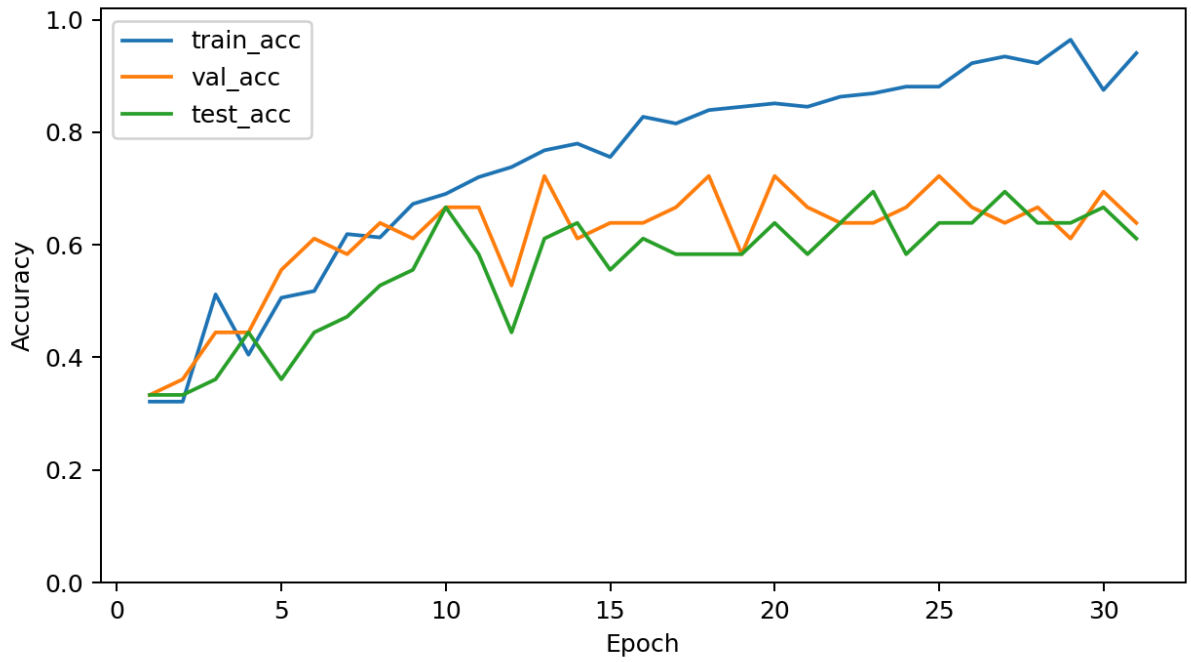

Figure 2. baseline64 accuracy curve. Validation accuracy peaked at 72.22%, but final test accuracy declined to 61.11%.

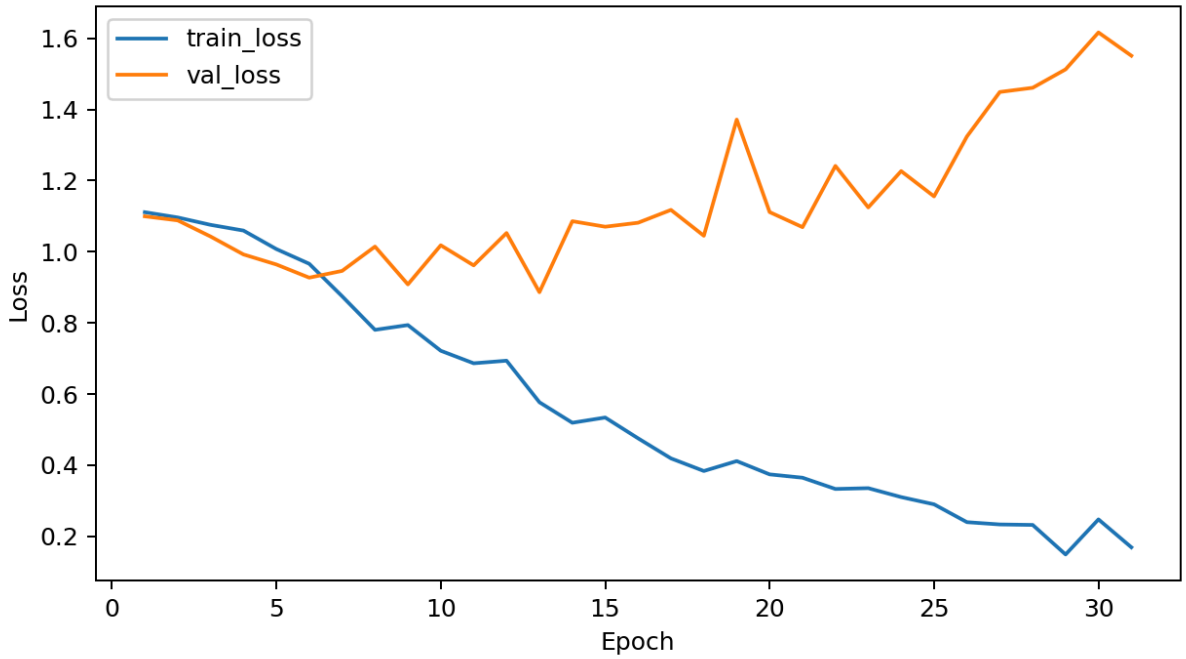

Figure 3. baseline64 loss curve. Training loss kept decreasing while validation loss increased after the early phase, indicating overfitting.

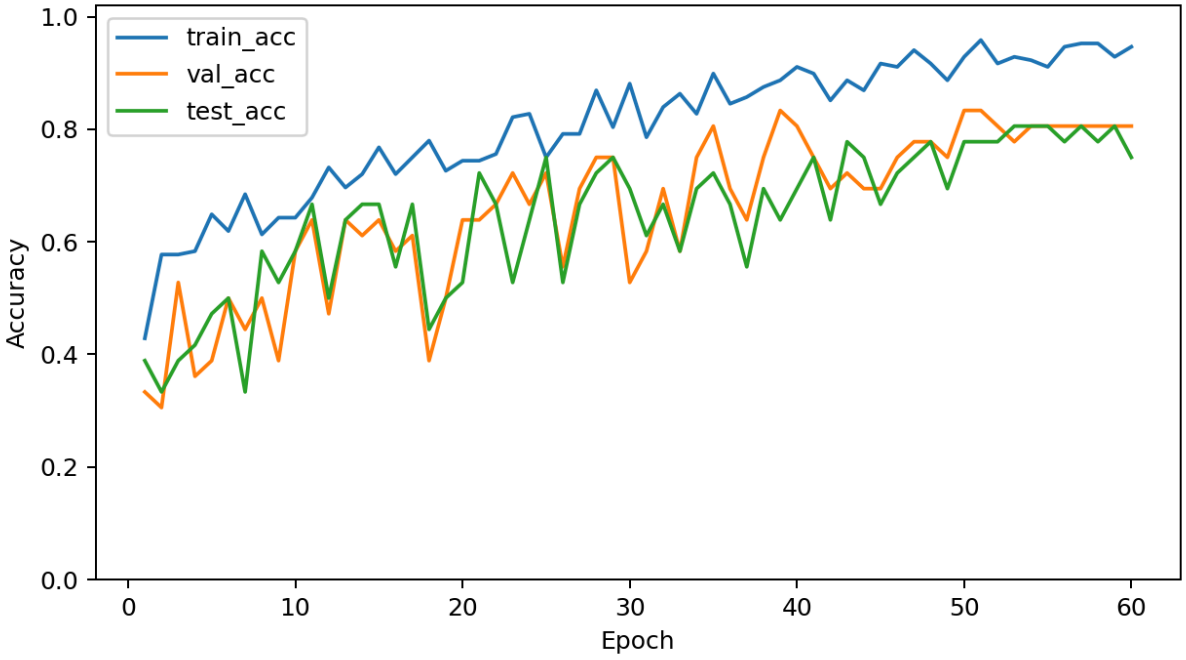

Figure 4. compact128 accuracy curve. This model reached 83.33% validation accuracy and 80.56% best test accuracy.

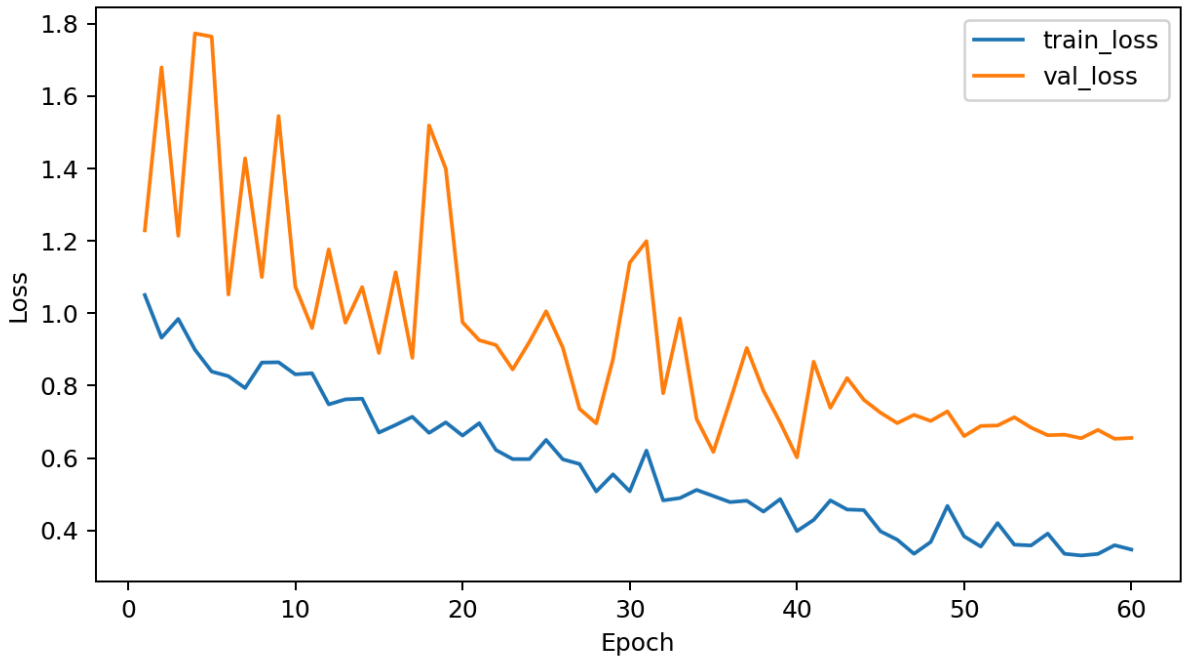

Figure 5. compact128 loss curve. Loss became more stable than the baseline due to stronger architecture and regularization.

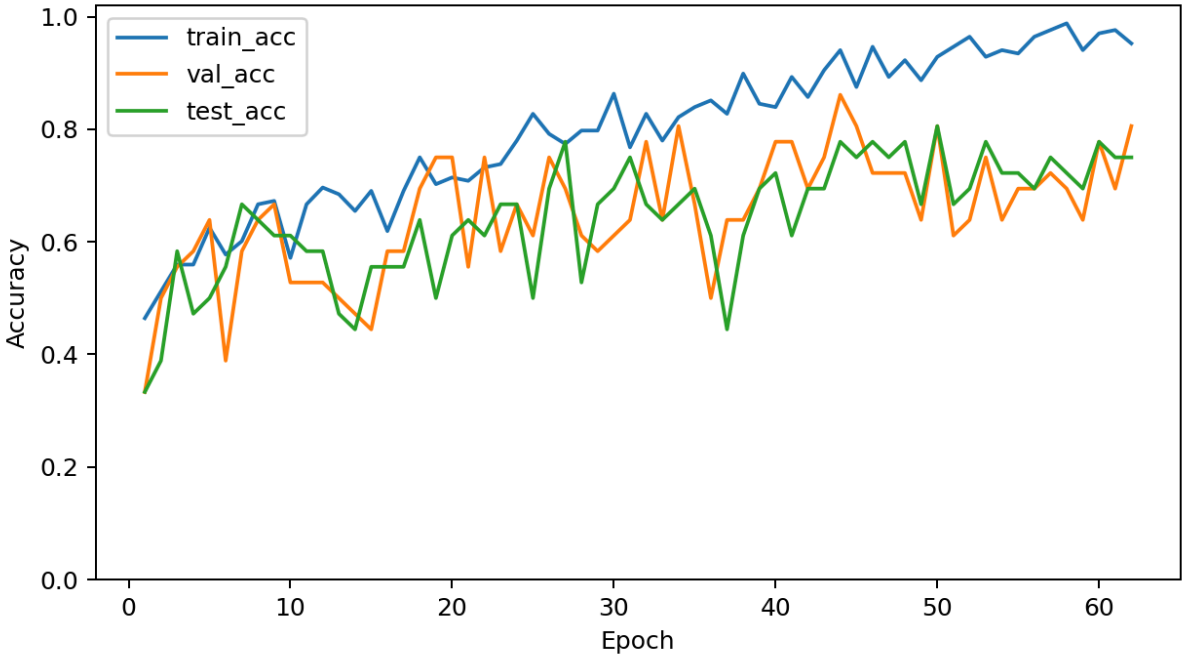

Figure 6. deeper160 accuracy curve. This model reached the highest validation accuracy, 86.11%, and also reached 80.56% best test accuracy.

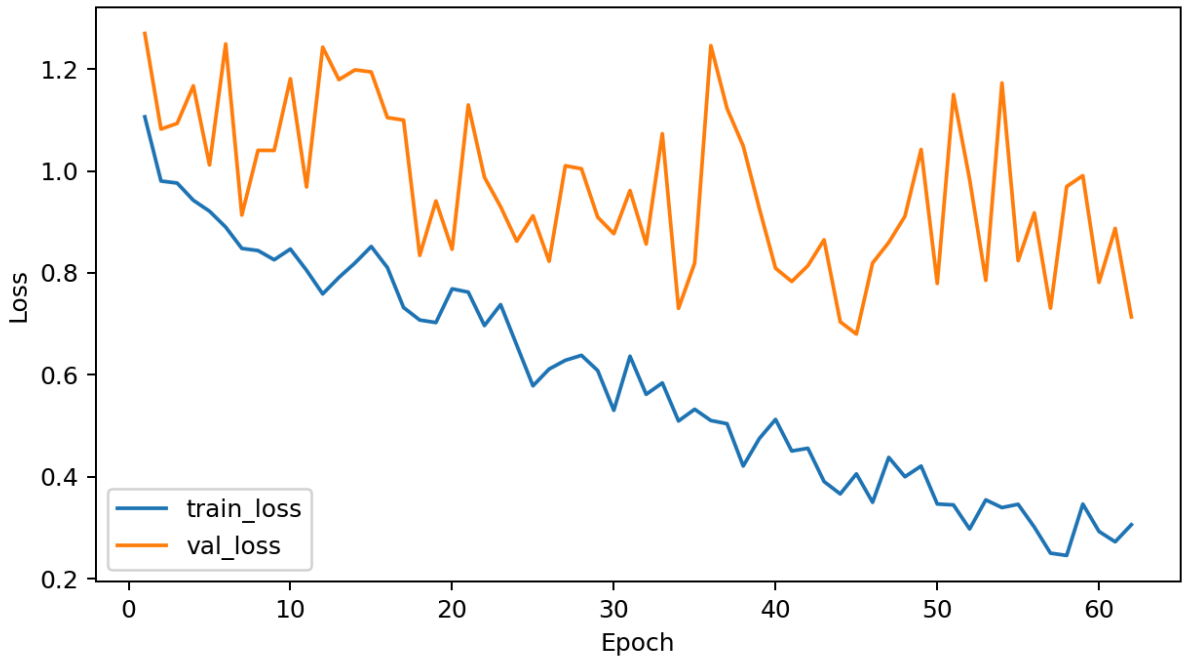

Figure 7. deeper160 loss curve. The model still shows validation fluctuation, but it provides the strongest fitting ability and best validation performance.

Baseline64 learned quickly but overfit. Its training accuracy rose to 94.05% by the final epoch, while validation and test accuracy fell to 63.89% and 61.11%. The increasing validation loss indicates that the model memorized training-specific visual features instead of learning general identity features.

Compact128 improved generalization. It reached 83.33% validation accuracy and 80.56% best test accuracy. The higher input resolution and BatchNorm blocks allowed the model to learn better face/person features than the baseline while AdamW and dropout reduced overfitting.

Deeper160 achieved the best validation accuracy at 86.11%. It also matched the best test accuracy of 80.56%. The deeper model has stronger feature extraction and benefits from the larger 160 x 160 input. However, its training accuracy reached 95.24% by the final logged epoch, so overfitting risk remains.

8. Prediction Examples

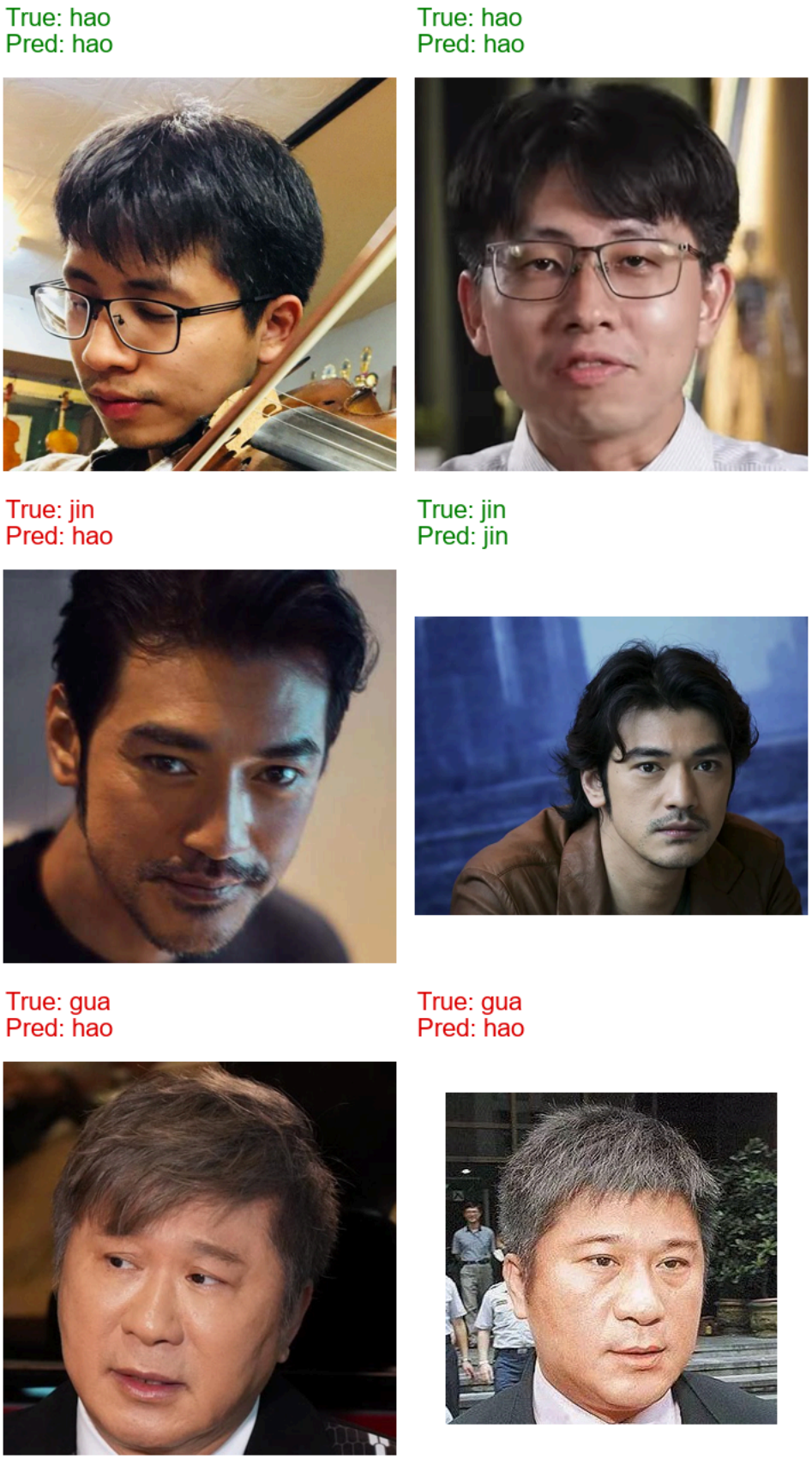

Figure 8. baseline64 prediction examples.

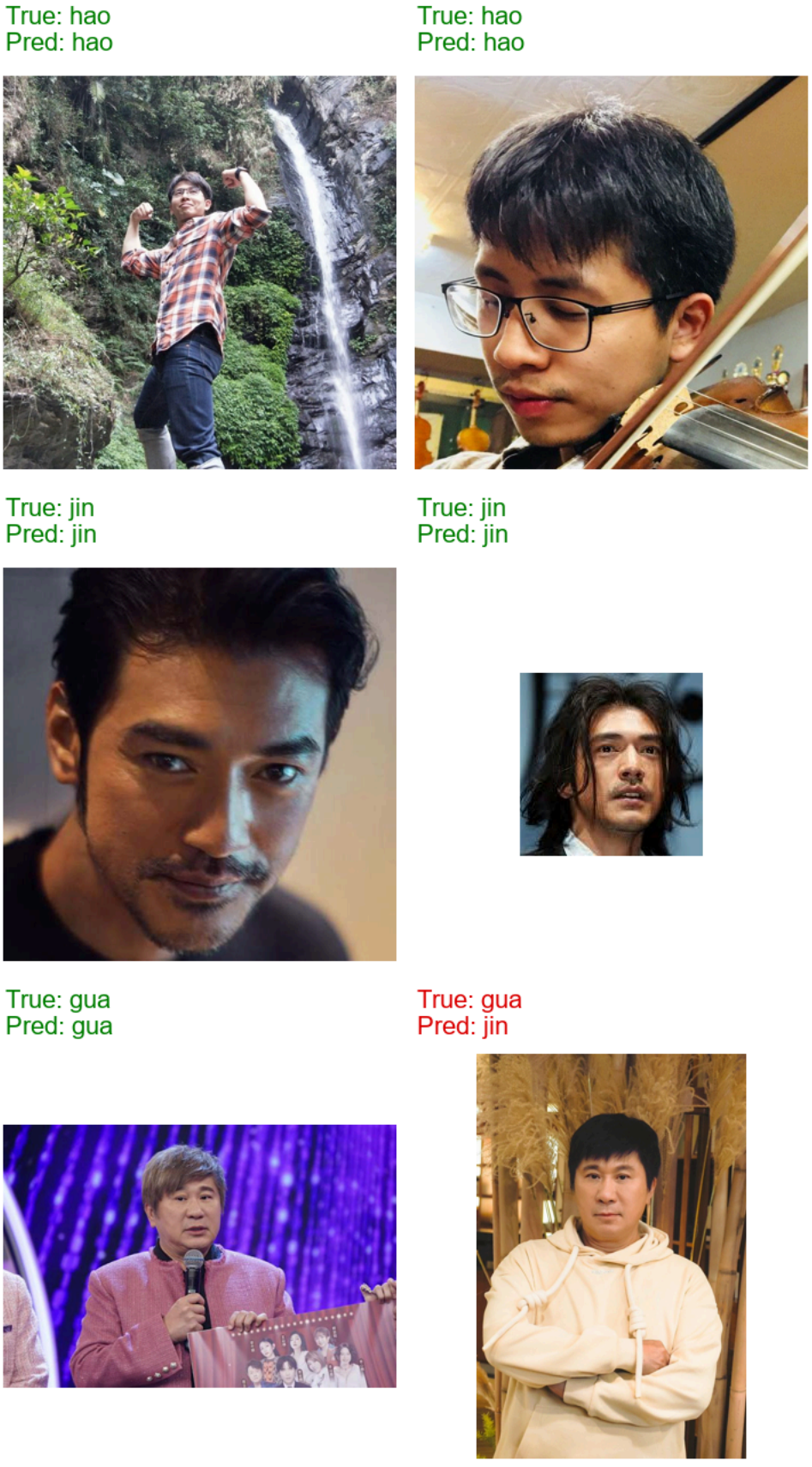

Figure 9. compact128 prediction examples.

True: hao
Pred: hao

True: hao
Pred: hao

True: jin
Pred: jin

True: jin
Pred: jin

True: gua
Pred: jin

True: gua
Pred: gua

Figure 10. deeper160 prediction examples.

True: hao
Pred: hao

True: hao
Pred: hao

True: jin
Pred: jin

True: jin
Pred: jin

True: gua
Pred: gua

True: gua
Pred: gua

Figure 11. deeper160-final (Full train, no val / test) prediction examples.

9. Best Model Selection and Final Training

The best model selected from the corrected local experiment histories is deeper160. The main reason is that it achieved the highest validation accuracy, 86.11%, and also reached the same best test accuracy as compact128, 80.56%.

- Highest best validation accuracy: 86.11% at epoch 44.
- Best test accuracy tied for first: 80.56%.
- Strongest architecture for mixed full-body and face-only inputs due to 160 x 160 input size and deeper ConvBN blocks.
- Compatible with final training on all available images before the hidden test submission.

After the experiment comparison, the final training command used all available images as training data and did not reserve validation/test data:

```
python -u train.py --raw_dir . --config deeper160 --final_train_all
--epochs 100
```

Because this final run uses all images for training, it does not produce validation or test accuracy. The formal performance comparison should therefore be based on the three experiment histories above. The final all-data run is used to create the strongest submission checkpoint for hidden TA evaluation.

10. Mock Hidden Test - Prediction Examples

The uploaded test_predictions.csv contains 12 predictions from the unlabeled hidden_test folder. The true_label column is empty, so this is not an official accuracy evaluation. The model matches 11 of 12 predictions, or 91.67%. This strong result support that this final best model could perform well for the hidden test or any other test/use.

Figure 8. Prediction examples for the mock hidden test. The true labels are unknown because it is a hidden test (no label).

hidden test image	pred - gua	pred - hao	pred - jin
gua	4	1	0
jin	0	0	7

Mock hidden test accuracy: 91.67% (11/12).

11. Notes, Limitations, and Submission Checklist

- The final model was trained on all available images, so final-run validation/test metrics are intentionally unavailable.
- The fixed class order must remain CLASS_NAMES = ["hao", "jin", "gua"].
- The code uses relative paths and saves weights under models/.
- The final checkpoint to submit should be **models/best_model.pth** from the final all-data deeper160 training run.

12. Conclusion

Overall, the experiments show that CNN performance is strongly affected by model depth, input image size, regularization, and preprocessing strategy. The baseline64 model provided a useful starting point, but its simpler architecture and smaller input size limited its ability to learn more detailed facial and body-related features, resulting in lower validation performance. The compact128 model improved feature extraction by using a larger input size, BatchNorm, stronger regularization, and training augmentation, which helped the model generalize better than the baseline. The deeper160 model further increased the model capacity and used higher-resolution inputs, allowing it to learn richer visual patterns from both full-body images and face-only images. However, the training and validation curves also show signs of fluctuation and possible overfitting, especially because the dataset size is relatively small and the validation/test sets contain limited samples. In conclusion, the improved CNN models performed better than the baseline, and the best-performing configuration was selected because it achieved the strongest validation performance while still maintaining reasonable test accuracy. For the final submission, I retrain the selected model on all available images so it allows the model to learn from the complete dataset before being evaluated on the hidden TA test set.